LeetCode HOT 100

PART 17

Tricks

技巧

96. 只出现一次的数字 (No.136) EASY

给你一个 **非空** 整数数组 `nums` ，除了某个元素只出现一次以外，其余每个元素均出现两次。找出那个只出现了一次的元素。

你必须设计并实现线性时间复杂度的算法来解决此问题，且该算法只使用常量额外空间。

示例 1：

> **输入：** nums = [2,2,1]
>
> **输出：** 1

示例 2：

> **输入：** nums = [4,1,2,1,2]
>
> **输出：** 4

示例 3：

> **输入：** nums = [1]
>
> **输出：** 1

提示：

- `1 <= nums.length <= 3 * 10^4`
- `-3 * 10^4 <= nums[i] <= 3 * 10^4`
- 除了某个元素只出现一次以外，其余每个元素均出现两次。

96. 只出现一次的数字 (No.136) EASY

思路

用异或位运算来消除重复出现的数字。核心利用3点：①任何数和自身异或都等于0；②任何数和0异或都等于自身；③异或运算具有交换律和结合律。那么只要初始化res=0，把nums中所有数异或一遍，遍历完成后的res就是那个没人和它异或的单独数字。

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        res = 0
        for num in nums:
            res ^= num
        return res
```

时间复杂度： $\mathcal{O}(n)$；**空间复杂度：** $\mathcal{O}(1)$

97. 多数元素 (No.169) EASY

给定一个大小为 `n` 的数组 `nums` ，返回其中的多数元素。多数元素是指在数组中出现次数 **大于** `⌊ n/2 ⌋` 的元素。

你可以假设数组是非空的，并且给定的数组总是存在多数元素。

示例 1：

> **输入：** nums = [3,2,3]
> **输出：** 3

示例 2：

> **输入：** nums = [2,2,1,1,1,2,2]
> **输出：** 2

提示：

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-10^9 <= nums[i] <= 10^9`
- 输入保证数组中一定有一个多数元素。

进阶： 尝试设计时间复杂度为 O(n)、空间复杂度为 O(1) 的算法解决此问题。

97. 多数元素 (No.169) EASY

思路

Moore投票法。台上只能有1个候选人res，对于每个num，如果num=res就投支持票vote+1，num≠res就投反对票vote-1，最后留在台上的res一定是那个多数元素。因为题目保证了多数元素的存在（一定超过半数），那么无论一开始res是谁，超过半数的多数元素一定能投足够多的反对票给他拉下来。因为每个元素眼中，其它元素都是反对票，想要成为候选人，必须自己要有足够多的反对票和其它所有人持平，还要留有一票自己上台，只有多数元素能做到这一点。

时间复杂度：$\mathcal{O}(n)$

空间复杂度：$\mathcal{O}(1)$

```python
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        res, vote = -inf, 0
        for num in nums:
            if not vote:  # 票数为0，更改候选人
                res = num
            vote += 1 if num == res else -1
        return res
```

98. 颜色分类 (No.75) MEDIUM

给定一个包含红色、白色和蓝色、共 `n` 个元素的数组 `nums` ，**原地** 对它们进行排序，使得相同颜色的元素相邻，并按照红色、白色、蓝色顺序排列。

我们使用整数 `0`、 `1` 和 `2` 分别表示红色、白色和蓝色。

必须在不使用库内置的 sort 函数的情况下解决这个问题。

示例 1：

> **输入：** `nums = [2,0,2,1,1,0]`
> **输出：** `[0,0,1,1,2,2]`

示例 2：

> **输入：** `nums = [2,0,1]`
> **输出：** `[0,1,2]`

提示：

- `n == nums.length`
- `1 <= n <= 300`
- `nums[i]` 为 `0`、`1` 或 `2`

进阶：

- 你能想出一个仅使用常数空间的一趟扫描算法吗？

98. 颜色分类 (No.75) MEDIUM

思路

这题荷兰国旗问题是[4.移动零]的进阶版。red和white的处理和[4.移动零]一模一样，遇到1不管，遇到0就和red交换，然后red自增。

不同的是有三种颜色，因此需要red和blue分别从左往右、从右往左维护下一个0和2放的位置。

三者下标red≤white≤blue，循环中始终保证：①red左侧都是0；②red到white之间都是1；③blue右侧都是2。

遇到2要拼命往后和blue交换，“拼命”意味着如果换回来还是2就还要继续换，换到不是2为止，这样保证把所有2都往后挪，因为根据上面①②，white不允许左侧有2。

遇到0就不需要拼命换，因为red到white之间只有1，不会再换回一个0。

```python
class Solution:
    def sortColors(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        # red是从左往右下一个0放的位置，blue是从右往左下一个2放的位置
        red, blue = 0, len(nums) - 1
        white = 0  # white负责把所有2往后移动到blue处，把0移动到red处
        while white <= blue:  # blue的位置还没处理过，是<=不是<
            # 当前位置是2，white不能往前走，防止blue处又换回来个2
            if nums[white] == 2:
                nums[white], nums[blue] = nums[blue], nums[white]
                blue -= 1
            elif nums[white] == 0:  # 当前位置是0，和red交换
                nums[white], nums[red] = nums[red], nums[white]
                red += 1
                white += 1
            else:  # 当前位置是1，符合white
                white += 1
        return nums
```

时间复杂度：$\mathcal{O}(n)$；空间复杂度：$\mathcal{O}(1)$

99. 下一个排列 (No.31) MEDIUM

思路

这题举几个例子就能发现下一个排列的规律，代码就是模拟了这个规律的过程。

①先从右往左找到第一个下降的位置i。

②此时i可能=-1，除此以外要交换i和i右侧比自己大的数中最小的那个数j。由于i右侧降序排列，只要找到第一个满足nums[i]≥nums[j+1]的数j即可，然后i和j处交换

③逆置i右侧的所有数。

时间复杂度：$\mathcal{O}(n)$

空间复杂度：$\mathcal{O}(1)$

```python
class Solution:
    def nextPermutation(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        n = len(nums)
        i = n - 2
        # step1:从右往左找到第一个下降的位置i
        while i >= 0 and nums[i] >= nums[i+1]:
            i -= 1
        # step2:除非整个nums递减，否则交换i和i右侧比自己略大的数
        if i >= 0:
            j = i
            # 由于i右侧递减，因此比自己略大的数j就是第一个满足nums[i]>=nums[j+1]的数
            while j < n - 1 and nums[i] < nums[j+1]:
                j += 1
            # 交换i和j处
            nums[i], nums[j] = nums[j], nums[i]
        # step3：逆置i右侧的所有数
        j, k = i + 1, n - 1
        while j < k:
            nums[j], nums[k] = nums[k], nums[j]
            j += 1
            k -= 1
```

100. 寻找重复数 (No.287) MEDIUM

给定一个包含 `n + 1` 个整数的数组 `nums` ，其数字都在 `[1, n]` 范围内（包括 `1` 和 `n`），可知至少存在一个重复的整数。

假设 `nums` 只有 **一个重复的整数** ，返回 **这个重复的数** 。

你设计的解决方案必须 **不修改** 数组 `nums` 且只用常量级 `O(1)` 的额外空间。

示例 1：

> **输入：** `nums = [1,3,4,2,2]`
> **输出：** `2`

示例 2：

> **输入：** `nums = [3,1,3,4,2]`
> **输出：** `3`

示例 3 ：

> **输入：** `nums = [3,3,3,3,3]`
> **输出：** `3`

提示：

- $1 <= n <= 10^5$
- `nums.length == n + 1`
- `1 <= nums[i] <= n`
- `nums` 中 **只有一个整数** 出现 **两次或多次** ，其余整数均只出现 **一次**

进阶：

- 如何证明 `nums` 中至少存在一个重复的数字？
- 你可以设计一个线性级时间复杂度 `O(n)` 的解决方案吗？

100. 寻找重复数 (No.287) MEDIUM

思路

这题作为HOT100收官题相当精彩。和[17.缺失的第一个正数]一样，运用了下标和元素映射的关系；做法上又和[26.环形链表II]一模一样，因此实现细节和原理不再赘述。

下标和元素映射后数组等价于一个链表，当前元素作为下一个元素的下标，就像next把链表串起来。通过鸽巢原理知道nums中必有重复数，重复数的存在导致不同节点的next可能是同一个元素，从而形成环。找重复数就是找环的入口下标。

```python
class Solution:
    def findDuplicate(self, nums: List[int]) -> int:
        slow, fast = 0, 0
        # 找到slow和fast的相遇点
        while True:  # 题目保证有环
            slow = nums[slow]
            fast = nums[nums[fast]]
            if slow == fast:
                break
        # 环的入口下标就是重复数
        p = 0
        while p != slow:
            p = nums[p]
            slow = nums[slow]
        return slow
```

时间复杂度：$\mathcal{O}(n)$

空间复杂度：$\mathcal{O}(1)$